Project - High Level Design

Multi-Agent Hospitality System

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	IRIFA KHAISHAGI	EN22CS301435
2.	AKASH GHOSH	EN22CS301082
3.	AASTHA GUPTA	EN22CS301019
4.	AMAN GOUR	EN22CS301104
5.	AESHNA PREYASI	EN22CS301075
	ADARSH DIBEY	EN22CS301037

Group Name: Group 08D1

Project Number: AAI-08

Industry Mentor Name: Aashruti Shah (Program Director, Datagami)

University Mentor Name: Prof. Nishant Shrivastav

Academic Year: 2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

1.1 Scope of the Document

This document covers the High-Level Design of the Multi-Agent Hospitality System. It describes the overall system structure, how the major components are organized, how data moves through the system, and what design decisions were made along the way. It does not go into code-level specifics; those are covered in the Low-Level Design document. The goal here is to give anyone reading this a clear picture of what the system does and how it is put together, before they look at any implementation details.

1.2 Intended Audience

This document is meant for academic evaluators, project mentors, and anyone on the team who needs to understand the system's structure before diving into the LLD or the code. It can also serve as a reference if the project is extended or handed off in the future.

1.3 System Overview

The Multi-Agent Hospitality System is a web application that takes a user's travel preferences - destination, number of days, and budget also produces a complete, structured day-wise travel itinerary. Instead of relying on a single AI model to do everything at once, the system breaks the task into stages and assigns each stage to a dedicated AI agent. These agents work in sequence, each building on the output of the previous one. The whole pipeline is managed by CrewAI, the frontend is built on Streamlit, and the Groq LLM API serves as the intelligence layer for all agents.

2. System Design

2.1 Application Design

The system is built around a layered architecture, which keeps different concerns cleanly separated and makes the codebase straightforward to maintain or extend.

The Presentation Layer is a Streamlit web interface where the user submits their travel details and views the final output. The Business Logic Layer sits behind it, this is where the CrewAI Orchestrator lives, responsible for coordinating all agents and managing the flow of the task. The Agent Layer is the core of the system, consisting of five agents: Research, Planner, Budget, Writer, and Reviewer, each handling one piece of the overall task. Every agent relies on the AI Layer, the Groq LLM API to generate its responses. Finally, the Utility Layer provides logging support and a DuckDuckGo-based search tool that the Research Agent uses to pull in real-world travel data.

Figure 1: High Level Architecture Diagram

2.2 Process Flow

The user starts by entering their travel details on the Streamlit interface. This input is picked up by the CrewAI Orchestrator, which kicks off the agent pipeline. The Research Agent goes first, gathering relevant information about the destination. That output flows into the Planner Agent, which drafts the day-by-day itinerary. The Budget Agent then takes that plan and works out the estimated costs. The Writer Agent formats everything into a readable structure, and finally the Reviewer Agent polishes it before the result is sent back to the UI for the user to see.

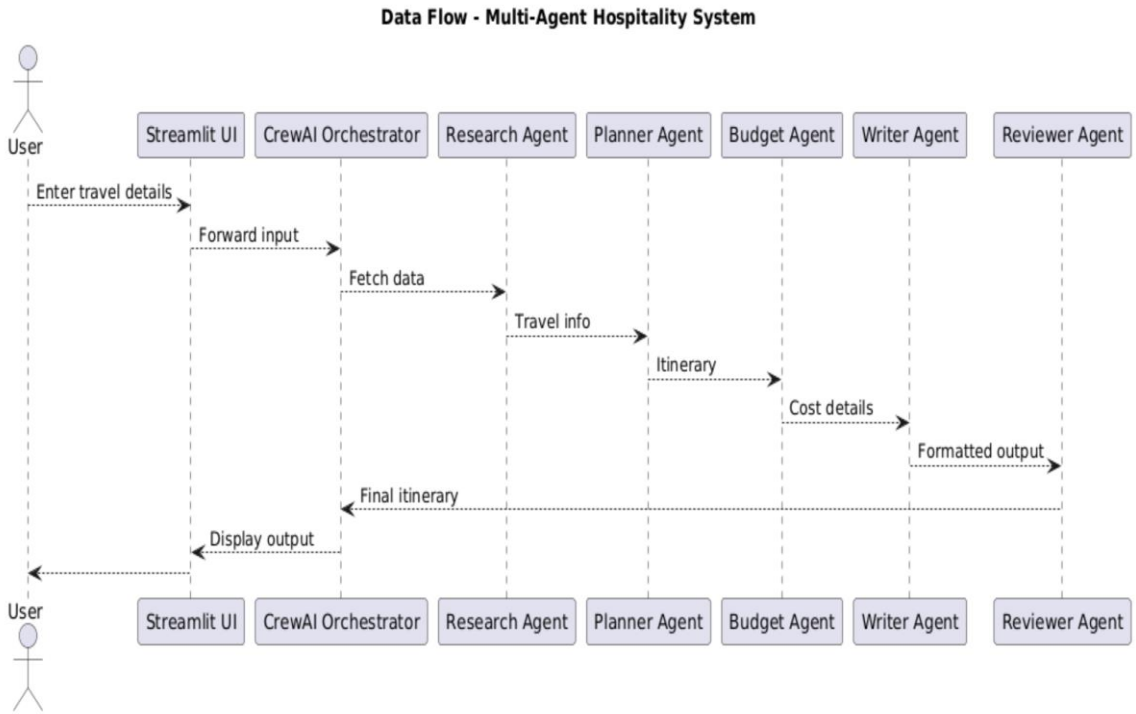

Figure 2: Process Flow Diagram

2.3 Information Flow

At each stage, the output of one agent becomes the input of the next. The user's original input: destination, duration, and budget travels all the way through the pipeline as context. The Research Agent's findings inform the Planner, the plan informs the Budget Agent, and so on. Nothing is discarded mid-pipeline; each agent gets the accumulated context it needs to do its job well. The final itinerary returned to the UI is the product of all five agents working together.

[Refer Figure 2]

2.4 Components Design

The system is made up of clearly defined, single-purpose components. Each component is responsible for exactly one part of the overall task, which keeps the system easy to debug and extend.

Component	Responsibility
Streamlit UI	Collects user input, renders the final itinerary
CrewAI Orchestrator	Manages agent execution order and data handoff
Research Agent	Gathers destination information and travel data
Planner Agent	Builds the day-wise itinerary

Budget Agent	Estimates costs based on the plan and budget
Writer Agent	Formats the itinerary into readable output
Reviewer Agent	Reviews and refines the final content
Groq LLM API	Powers all agents with language generation
DuckDuckGo Search API	Provides real-world data to the Research Agent
Logger	Writes activity and error logs to app_logs.txt

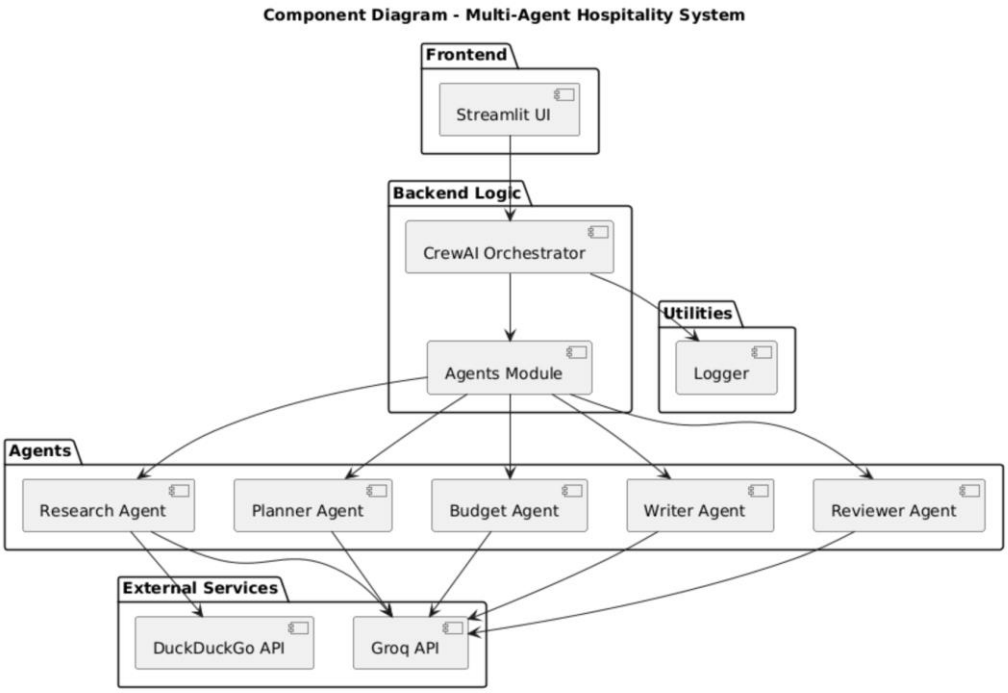

Figure 3: Component Diagram

2.5 Key Design Considerations

The main design principle behind this system is that each agent should do one thing and do it well. This makes the pipeline easy to debug, if the budget estimates are off, you look at the Budget Agent; if the formatting is wrong, you look at the Writer Agent. It also means the system can be extended later; adding a new capability just means adding a new agent rather than reworking the whole flow.

The layered architecture keeps the UI, the business logic, and the AI integration independent of each other. Logging was included from the start so that system behavior can be tracked and errors can be diagnosed without having to reproduce issues manually. The choice to use the Groq LLM for all agents rather than mixing models keeps the system consistent and avoids compatibility issues.

2.6 API Catalogue

Groq LLM API: The primary intelligence provider. Used by all five agents to generate responses at each stage of the pipeline. Requires an API key for authentication.

DuckDuckGo Search API: Used by the Research Agent to fetch current, real-world information about the travel destination. No strict authentication is required.

CrewAI Internal API: Manages communication between agents, handles task assignments, and ensures the sequential flow of the pipeline is maintained correctly.

3. Data Design

3.1 Data Model

The system does not use a database. All data is held in memory during a session. The primary data object flowing through the pipeline is the user's travel request, which carries the destination, number of days, and budget. As each agent processes it, the object grows first gaining research findings, then an itinerary structure, then cost estimates, then formatted content, and finally the reviewed output.

3.2 Data Access Mechanism

Since there is no persistent database, data access in this system means passing structured text between agents via the CrewAI framework. The Orchestrator handles this, each agent receives its required inputs as part of its task definition and returns its output as text, which the Orchestrator then passes forward.

3.3 Data Retention Policies

The system does not retain user data between sessions. Once a session ends and the browser is closed, the generated itinerary is gone unless the user saves it manually. The only persistent data the system writes is the activity log in app_logs.txt, which records system events and any errors encountered during execution.

3.4 Data Migration

Data migration is not applicable to the current version of this system, as there is no database and no stored user history. If a future version introduces persistent storage or a user account system, a migration strategy would need to be defined at that point.

4. Interfaces

User Interface: The Streamlit web app is the only user-facing interface. It presents input fields for destination, number of days, and budget, and displays the generated itinerary once the pipeline completes. It is browser-based and runs locally.

Groq LLM API Interface: Each agent communicates with the Groq API via standard HTTP requests, sending a prompt and receiving a generated text response. The API key is loaded from the environment.

DuckDuckGo Search Interface: The Research Agent uses the DuckDuckGo Search Tool wrapper provided within CrewAI to query travel information. Results are returned as text snippets that the agent processes further.

Internal Agent Interface: Agents communicate indirectly through the CrewAI Orchestrator. There is no direct agent-to-agent communication; all data handoffs go through the orchestration layer.

5. State and Session Management

The system is stateless by design. Each time a user submits a request, a fresh pipeline execution begins from scratch. There is no user authentication, no stored session history, and no memory carried over from one run to the next.

Within a single run, state is managed implicitly by the CrewAI Orchestrator, it tracks which agent has completed its task and what output to pass to the next agent. Streamlit handles its own UI state using its built-in session state mechanism, ensuring that the displayed output remains visible until the user submits a new request or refreshes the page.

6. Caching

The current version of the system does not implement explicit caching. Every request triggers a full pipeline run, including fresh API calls to Groq and, if used, the DuckDuckGo search tool.

That said, Streamlit provides a built-in caching decorator that could be used in future iterations to cache repeated queries, for example, if a user runs the same destination multiple times. This would reduce response time and unnecessary API usage without requiring a separate caching layer.

7. Non-Functional Requirements

7.1 Security Aspects

The Groq API key is the only sensitive credential in the system and should be stored in environment variables rather than hardcoded in the source. Since the system runs locally with no user accounts or stored personal data, the attack surface is minimal. No sensitive user information is transmitted or retained beyond the active session.

7.2 Performance Aspects

Response time depends primarily on how long the Groq LLM takes to respond across five sequential agent calls. Since each agent waits for the previous one to finish, the total latency is roughly the sum of all five API round-trips. Using lightweight models via Groq keeps this manageable for a local setup. There is no background processing, no queuing system, and no parallel execution in the current design, the pipeline runs synchronously from start to finish.

8. References

- Official CrewAI documentation - agent orchestration, task definitions, and workflow design
- Official Streamlit documentation - building and running interactive web applications
- Groq API documentation - model selection, authentication, and API usage
- Official Python documentation - standard libraries and programming best practices
- Research articles and online resources on multi-agent system design patterns